



Sensorveiledning: Millions

IDATx2003 Programmering 2, våren 2026

Innhold

<i>Om emnet</i>	<i>1</i>
Emnebeskrivelse	1
Lærestoff og verktøy/IDE	1
<i>Om mappen.....</i>	<i>2</i>
Mappens innhold.....	2
Krav og betingelser.....	2
Spesifikt om bruk av KI	3
<i>Karakterskalaen ved NTNU</i>	<i>4</i>
<i>Sensursskjema.....</i>	<i>5</i>
Del 1 – Funksjonelle krav (15%)	6
Del 2 – Implementasjon/Design (65%).....	7
Del 3 – Rapport (30%).....	13
<i>Illustrasjoner</i>	<i>15</i>
<i>Vedlegg</i>	<i>16</i>

Om emnet

Emnebeskrivelse

Lenke til formell emnebeskrivelse: <https://www.ntnu.no/studier/emner/IDATA2003>

Emnet er felles på tvers av campusene Gjøvik, Ålesund og Trondheim, med egen emnekode per campus: IDATG2003/IDATA2003/IDATT2003. Vi refererer til emnet som IDATx2003.

IDATx2003 Programmering 2 dekker følgende læringsutbytter i OOP:

- Byggesystemer (Maven)
- Arv og polymorfi
- GUI (JavaFX)
- Datafiler og serialisering
- Unntakshåndtering
- Designmønster

I tillegg gjelder også læringsutbytter fra IDATx1003 Programmering 1:

- Samarbeid mellom objekter, lister, søk og sortering, sammenligning av objekter, integrerte utviklingsmiljø, debugging.
- Funksjonell programmering
- Enhetstesting
- Versjonskontroll

Lærestoff og verktøy/IDE

Ved campusene Gjøvik og Ålesund benyttes læreboka «Objects First with Java» av David J. Barnes og Michael Kölling. Campus Trondheim baserer seg hovedsakelig på nettressurser som leksjoner, presentasjoner, videoer og kodeeksempler. Faget har også en wiki.

Vi viderefører profesjonelle utviklingsmiljø som IntelliJ og VS Code fra IDATx1003 Programmering 1. Studentene står fritt til å velge IDE selv. Vi introduserer også Maven fra start, slik at samtlige eksempler og oppgaver studentene jobber med gjennom semesteret er Maven-prosjekter.

Git og GitHub/GitLab videreføres fra IDATx1003 Programmering 1.

Om mappen

Mappens innhold

Mappen som er levert til sensur består av:

- En prosjektoppgave i programmering som kandidatene har jobbet med i grupper på to gjennom semesteret. Gruppene har fått muntlig tilbakemelding på arbeidet sitt tre ganger, i tillegg til veiledning ved behov. *Teller 70% av endelig karakter.*
- En rapport, der gruppen skal beskrive løsningen, hvilke valg som er gjort under utviklingen av løsningen, og hvilke endringer (refaktoring) som er utført. Gruppen skal også begrunne designet sitt basert på teori rundt for eksempel coupling, cohesion og responsibility driven design, med konkrete eksempler fra koden. *Teller 30% av endelig karakter.*

I årets prosjekt har studentene utviklet et aksjespill. Prosjektoppgaven har blitt presentert til studentene etappevis gjennom semesteret:

- Del 1 ble publisert i uke 5, med fokus på grunnleggende klasser og logikk, i tillegg til oppsett av prosjektet.
- Del 2 ble publisert i uke 9, med fokus på filhåndtering og støtte for statistikk og status.
- Del 3 ble publisert i uke 13, med fokus på GUI, designmønstre og utvidelser basert på egne ideer.

Krav og betingelser

Aksjespillet har følgende funksjonelle krav:

- Det skal være mulig å starte et nytt spill. Ved oppstart skal brukeren kunne angi navn, startkapital, og en vilkårlig fil med aksjedata:
 - Brukeren må selv kunne velge fil fra hvor som helst på datamaskinen
 - Programmet må støtte formatet som er angitt i mappens del 2
- Det må være mulig å hente ut informasjon om aksjene på børsen:
 - Systemet må tilby søk, filtrering e.l. som gjøre det lett å finne enkeltaksjer
 - Det skal være mulig å hente ut statistikk om hver aksje (høyeste og laveste pris, prishistorikk og så videre)
 - Programmet skal kunne vise info om ukens vinnere og tapere
- Det skal være mulig å kjøpe og selge andeler:
 - Spillerens portefølje må være lett synlig, slik at man får et overblikk over hvilke andeler man eier og kan selge
 - Før man fullfører en transaksjon bør man få informasjon om verdier og kostnader knyttet til transaksjonen
 - Etter at man har fullført en transaksjon må systemet vise en kvittering (transaksjonsrapport)
- Gjennomførte transaksjoner må være tilgjengelige:
 - Det bør være enkelt å søke/filtrere i en transaksjonssoversikt
 - Det må være mulig å få detaljert info om hver enkelt transaksjon (kvantitet, pris, kostnader og så videre)
- Når man er ferdig med ukas handel må man kunne avansere til ny uke:

- Dette medfører endring i priser, som naturlig nok må gjenspeiles ellers i programmet
- GUI må opplyse om spillerens nettoverdi og status i sanntid
- Det skal være mulig å selge unna hele beholdningen og avslutte programmet

GUI skal designes i henhold til MVC og være brukervennlig og responsivt. I tillegg skal studentene vise at de behersker utvalgte designmønstre (factory og observer), samt utvide løsningen basert på egne ideer.

Andre viktige betingelser:

- Maven skal benyttes som byggesystem, og man må kunne bygge og kjøre applikasjonen med Maven-kommandoer fra terminalen
- Prosjektet skal være underlagt versjonskontroll med Git og GitHub/GitLab
- Koden skal ha enhetstester for den delen som er forretningskritisk
- Unntak må håndteres på en robust måte med beskrivende feilmeldinger
- Det forventes høy kodekvalitet i prosjektet
- Studentene velger selv hvilket IDE de ønsker å benytte

Se prosjektoppgaven (vedlegg) og sensurskjema (under) for en fullstendig beskrivelse av krav, betingelser og vurderingskriterier.

Spesifikt om bruk av KI

Det er tillatt å anvende KI-verktøy som støtte for å løse mappen, så lenge

- verktøyene først og fremst benyttes som sparringspartner/rådgiver og oppslagsverk
- eventuell kode som foreslås gjennomgås kritisk/bearbeides før bruk
- studentene kommenterer i koden der de har fått hjelp av slike verktøy
- bruken dokumenteres i rapporten (hvilke verktøy ble benyttet til hva, gjerne med eksempler på prompts og svar)

Karakterskalaen ved NTNU

Karakterbeskrivelser for emner i teknologiske fag ved NTNU:

Symbol	Betegnelse	Generell, ikke fagspesifikk beskrivelse av vurderingskriterier
A	Fremragende	Kandidaten viser særdeles god kunnskap i og oversikt over emnets faglige grunnlag og innhold. Kandidaten viser meget stor grad av selvstendighet og solid analytisk forståelse. Kandidaten viser svært gode ferdigheter i anvendelsen av denne kunnskapen.
B	Meget god	Kandidaten viser meget god kunnskap i og oversikt over emnets faglige grunnlag og innhold. Kandidaten viser betydelig grad av selvstendighet og god analytisk forståelse. Kandidatens ferdigheter i anvendelsen av denne kunnskapen ligger over gjennomsnittet.
C	God	Kandidaten viser god oversikt over de viktigste kunnskapselementene og sammenhengene i emnets faglige grunnlag og innhold. Kandidaten viser selvstendighet. Kandidaten viser analytisk evne og forståelse. Kandidaten viser gjennomsnittlig evne til å anvende sin kunnskap. Gjennomsnittet kan dels tolkes som typisk prestasjon av mange studenter i emnet og dels som krav til tilfredsstillende god prestasjon i emnet.
D	Nokså god	Kandidaten viser i noen grad analytisk evne og forståelse. Kandidaten viser selvstendighet i noen grad. Kandidaten viser oversikt over de viktigste kunnskapselementene og sammenhengene i emnets faglige innhold, men denne oversikten er preget av noen vesentlige mangler. Kandidaten viser i noen grad evne til å bruke kunnskapen aktivt, men prestasjonen er noe dårligere enn gjennomsnittet.
E	Tilstrekkelig	Kandidaten viser mangelfull analytisk evne og forståelse. Kandidaten viser gjennomgående noe, men sporadisk preget oversikt over de viktigste kunnskapselementene og sammenhengene i emnets faglige innhold. Kandidatens prestasjon oppfyller minimumskravet som stilles i emnet når det gjelder kunnskap, analytisk evne og ferdighet i å anvende emnets kunnskapsinnhold.
F	Ikke bestått	Kandidatens prestasjon faller under minimumskravet som stilles i emnet når det gjelder kunnskap, analytisk evne og ferdighet i å anvende emnets kunnskapsinnhold.

Sensursskjema

Under følger sensursskjemaet som er lagt til grunn for sensurering av mappene ved alle tre campus (Trondheim, Gjøvik, Ålesund).

Poeng og vekting er ikke oppgitt i denne veiledningen. Det vil alltid gjøres en helhetsvurdering av kandidatens besvarelse før endelig karakter settes.

Sensursskjema er delt inn i tre deler:

- Del 1 – Funksjonelle krav, GUI og brukervennlighet, samt kreativ utvidelse (15%)
- Del 2 – Implementasjon/kode/løsning (65%)
- Del 3 – Rapport (30%)

Totalt kan kandidaten oppnå 110% grunnet en 10% ekstra bonus som kan benyttes om man mener besvarelsen inneholder ekstra prisverdige elementer.

Del 1 – Funksjonelle krav (15%)

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Funksjonelle krav				
• Det skal være mulig å starte et nytt spill. Ved oppstart skal brukeren kunne angi navn, startkapital, og en vilkårlig fil med aksjedata. • Det må være mulig å hente ut informasjon og statistikk om aksjene på børsen. • Det skal være mulig å kjøpe og selge andeler. • Gjennomførte transaksjoner må være tilgjengelige. • Når man er ferdig med ukas handel må man kunne avansere til ny uke, med endring i priser. • GUI må opplyse om spillerens nettoverdi og status i sanntid. • Det skal være mulig å selge unna hele beholdningen og avslutte programmet. • Egne utvidelser.	Applikasjonen kan ikke kjøres med GUI. Få eller ingen av kravene er oppfylt.	Svært få av de funksjonelle kravene er implementert.	De fleste og viktigste funksjonelle kravene er implementert.	Samtlige funksjonelle krav er oppfylt. I tillegg er løsningen utvidet basert på egne idéer (se også kriteriet om kreative utvidelser).
Brukervennlighet/GUI				
Prosjektet skal leveres med et grafisk brukergrensesnitt.				
Følgende skal vurderes i forhold til brukervennlighet og design av brukergrensesnitt:				
• GUI er intuitiv/oversiktlig • God informasjon til bruker • Applikasjonen presenteres på en oversiktlig måte • Fornuftige feilmeldinger • Robust (applikasjonen krasjer ikke ved «feil bruk»)	• Har ikke implementert GUI	• Et minimalt GUI • Mangler funksjonalitet via GUI • Mangler vesentlig informasjon • Uklart hvordan man går frem • Mangelfull informasjon til bruker • Håndterer ikke feilsituasjoner/unntak	• Helt grei GUI, viser de viktigste elementene • Kan utføre de viktigste funksjonene via GUI • Har minimalt med padding og insets (luftigere design) • Har noe håndtering av feil/unntak • Benytter basis-komponentene i JavaFX (label, button, text field osv) på en grei måte	• Svært brukervennlig GUI (oversiktlig, informativt, tydelig) • Kan utføre samtlige operasjoner via GUI • Håndterer feilsituasjoner/unntak svært godt (med dialoger/informasjon) • Benytter avanserte JavaFX-komponenter (som f.eks. TableView, TreeView, Mediaavspiller osv) • Responsiv layout (fungerer godt i ulike oppløsninger/skjemstørrelser)

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Kreative utvidelser				
Er applikasjonen utvidet basert på egne kreative idéer?	Har ikke utvidet	Har forsøkt å utvide, men ikke lyktes	Har utvidet med noen "kreative" elementer	Har utvidet med flere nye elementer

Del 2 – Implementasjon/Design (65%)

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Byggsystem/Maven				
Er pom.xml satt opp riktig?	Har ikke benyttet Maven	Har en minimum pom.xml fil med mangler/svakheter. F.eks: • inneholder syntaksfeil • er ugyldig • følger ikke konfigurasjonskravene i oppgaven (versjoner mm)	• pom.xml er satt opp riktig, med nødvendige dependencies og plugins for å bygge og kjøre.	Har i tillegg, for eksempel: • Generering av JavaDoc • Brukt properties for å håndtere versjoner • Dokumentert pom-fila med kommentarer for økt lesbarhet
Mappestruktur	Har ikke fulgt Maven mappestruktur	Følger noenlunde anbefalt mappestruktur, men: • Har ikke test-klassene i egen test-mappe • Har ikke ressurser i resources-mappen	Følger standard Maven mappestruktur for src og test, med grei bruk av pakker.	Har i tillegg • en logisk katalogstruktur for ressursfiler (f.eks. bildefiler i egen resources-mappe). • Skiller klart på filer som er ressurser og filer som er å betrakte som «brukerfiler».
Prosjektet bygger, testes og kan kjøres fra Maven.	Har ikke benyttet Maven	Prosjektet kompilerer • mvn compile - OK • men en eller flere av følgende feiler: • mvn test - FAIL (ingen av testene blir utført) • mvn package - FAIL • mvn javafx:run - FAIL	• mvn compile - OK • mvn test - OK (men tester kan feile) • mvn package - OK • mvn javafx:run - OK	• mvn compile - OK • mvn test - OK (ingen tester feiler) • mvn package - OK • mvn javafx:run - OK

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Versjonskontroll				
Lokalt/sentralt repo, commits og branching	Har ikke benyttet versjonskontroll	• Prosjektet er lagt til versjonskontroll lokalt • Sporadiske innsjekkinger (commits) - mye som er endret mellom hver commit • Mangelfulle commit-meldinger (enten tomme, eller ikke beskrivende for endringene som er utført) • Ikke sentralt repo	• Prosjektet har sentralt repo (GitHub/GitLab) • Fornuftig jevnlig innsjekking (commit) av endringer • Gode commit-meldinger som beskriver kort hvilke endringer som er gjort/hvilke problem som er løst • OK om greiner (branches) ikke er benyttet	• Har benyttet greiner som del av arbeidsflyt (f.eks. develop/main), for features/utprøving og liknende. • Har gjennomført merge mellom greiner • Har benyttet tags for å merke versjoner
Filer lagt til versjonskontroll	Har ikke benyttet versjonskontroll	• Benytter IKKE .gitignore • Har lagt unødvendige filer/mapper til versjonskontroll, som /target, /.idea, *.iml osv.	• Benytter .gitignore • Har filtrert bort de fleste filer og mapper	• Kun Maven-spesifikke filer er lagt til versjonskontroll (alle IDE-spesifikke filer er filtrert bort) • Har opprettet README.md-fil som gir en kort beskrivelse av prosjektet, og info om hvordan bygge og kjøre applikasjonen
Arv, polymorfi				
		• Har i noen grad implementert arv i henhold til oppgaveteksten • Viser noe begrenset forståelse for arv og polymorfi (f.eks. ved ikke å bruke variabler av superklasse/abstrakt klasse/interface for å holde på objekter av konkret klasse der dette ville ha vært naturlig) • Dupliserer kode i sub-klasser som burde vært lagt til superklasse	Viser grunnleggende forståelse for arv • sub/super klasser • polymorfi • typecasting • instanceof • osv • Fornuftig bruk av arv («is a»)	Demonstrerer svært god forståelse for arv og polymorfi, ved f.eks.: • Har innført egne hensiktsmessige superklasser og/eller interface utover oppgavebeskrivelsen • Bruker konsekvent Interface/super-typer for variabler og retur fra metoder, der dette måtte passe.

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Enhetstesting				
	Har ingen enhetstester	• Har noen enhetstester/tester noen få av klassene • Mangler negative tester/har ikke forstått "negativ test" • Skriver tester som medfører at tester må kjøres i en bestemt rekkefølge • Ikke gode beskrivende navn på testene	• Har gode beskrivende navn på testene • Følger prinsippet om Arrange-Act-Assert • Har enhetstester for de grunnleggende klassene • Har helt greie negative tester (viser at kandidaten har forstått hovedpoenget med positive/negative tester)	• Har svært gode negative tester for alle klassene som testes • Tester Exceptions • Benytter seg av "@BeforeAll", "@BeforeEach" osv - der det er hensiktsmessig • For test av lesing fra fil testes kun parsing, ikke filaksess
Lambda-uttrykk og Streams				
	Benytter ikke lambda eller streams	Minimal bruk av lambda	Benytter lambda i forbindelse med event-håndtering i GUI	• Benytter lambda i samlinger (forEach()), i stedetfor while/for-løkker, hvor det er hensiktsmessig • Øvrig "smart" bruk av lambda og streams
Grafisk brukergrensesnitt (GUI) med JavaFX				
Bruk av layout-panes	Ikke benyttet seg av panes for å strukturere innholdet på GUI (plassering av komponenter)	• Har «hardkodet» plassering av GUI komponenter (knapper, etiketter, tekstfelt osv) • All kode for å bygge GUI ligger i start()-metoden (ingen delegering)	• Har grei bruk av ulike panes og bokser for å plassere GUI-komponentene. • Grupperer komponenter som «hører sammen» • Laget submetoder/delegeringsmetoder for å håndtere byggingen av deler av GUI • Noe bruk av gap og insets for å skape litt luft i GUI	• Svært god bruk av ulike panes og bokser for å lage et skalerbart og brukervennlig GUI • Benytter seg av hjelpemetoder for å bygge opp de ulike delene av GUI • Innført egne klasser for deler av GUI
Model-View-Controller (MVC)	Har ikke anvendt MVC	• Har implementert en kontroller-klasse, men har ikke forstått MVC	• Grei implementasjon av MVC der kontroller-klassen har et tydelig og riktig ansvar. • Tydelige relasjoner mellom Controller og View-klassen(e), og hva som representerer Model.	• Svært god implementasjon av MVC. • Ingen logikk (annet en rent visuelt/ui som å forhindre inntasting av tekst i et tall-felt) utføres i view-rollen • Egne kontroll-klasser for separate deler av GUI der dette er hensiktsmessig

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Forståelsen av JavaFX – underliggende konsept samt håndtering av hendelser (events)	Har ikke forstått strukturen bak JavaFX (scene, stage osv)	• Viser en minimal grunnleggende forståelse av hvordan en scene-graph bygges • Enkel håndtering av hendelser (knapper)	• Viser god kontroll på å sette opp en scene-graph • Demonstrerer god bruk av hendelseshåndtering (knapper og menyer f.eks.) • Benytter til dels «avanserte» funksjoner for å skape et innbydende brukergrensesnitt (alignment, gap, insets m.m.)	• Demonstrerer svært god kontroll over JavaFX • Benytter avanserte funksjoner i JavaFX for å skape et godt, intuitivt brukergrensesnitt • Avansert håndtering av hendelser (som f.eks. sjekk av verdier i felt, mus-hendelser, fange endring av vindustørrelse m.m.)
Filhåndtering				
	Ingen filhåndtering	Leser kun fra forhåndslaget tekstfil fra hardkodet sti.	• Leser fra tekstfil fra vilkårlig sti • Begrenset sjekk/kontroll av filformat/struktur • Enkel håndtering av unntak (f.eks. skriver til System.out/System.err) • Kan skrive til tekstfil • Bruker både io- og nio-biblioteket • Lukker filressurser på en trygg måte • Delt opp lesing- og skriving i egne klasser • Separerer lesing/skriving fra parsing/koding av innhold	Har i tillegg: • Solid kontroll/sjekk av filformat/struktur på innhold. • Svært god unntakskontroll: fanger unntak lokalt for logging (ved bruk av logger og ikke System.out/err). Kaster unntak videre. • Har egne unntaksklasser (fortrinnsvis av typen checked) • Har designet lesing fra fil på en generisk måte som gjør det enkelt å innføre andre filformater senere.
Designmønster				
	Har ikke brukt designmønstre	Feil bruk av mønster, som viser tydelig at kandidaten ikke helt har forstått designmønster	• Har implementert designmønstre som er foreslått i oppgaven (Factory og Observer) • Har en fornuftig implementasjon av Observer-mønsteret i forhold til oppgaven • Enkel varsling til observatørene (update(), notify())	• Har anvendt andre designmønstre utover de som er foreslått i oppgaveteksten, på en god måte • Svært god kombinasjon av ulike designmønstre der dette er nyttig • Svært godt implementert Observer-mønster.

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Kodekvalitet/Design				
Er koden godt dokumentert iht JavaDoc-standard?	Ingen dokumentasjon	- Sporadisk dokumentasjon av klassene - Sporadisk dokumentasjon av metoder - Overfladisk dokumentasjon som ikke gir nyttig info ("A FileWriter writes files"), eller dok som beskriver hvordan og ikke hva - Mangelfull/fraværende bruk av "@param", "@return"	- Samtlige klasser er dokumentert med god beskrivelse av rolle/ansvar. - De fleste metoder som er public er godt dokumentert. - Grei "ordlyd" i dokumentasjonen - Noe bruk av "@param" og "@return"	- Gjennomført høy kvalitet i dokumentasjon - God formulering som klart dokumenterer klasser og metoder "@param", "@return" konsekvent brukt - Innslag av HTML-formatering for bedre lesbarhet hvis nødvendig
Er koden robust (verifiseres parametere før de brukes, er unntakshåndteringen solid m.m.)?	- Ingen verifisering - Ingen unntakshåndtering	- Sporadisk eller ingen sjekk av parameterverdier - Begrenset bruk av unntakshåndtering	- Gjennomgående grei verifisering av parameterverdier - Kaster og fanger unntak	- Verifiserer samtlige parametere - Har laget egne unntaksklasser
Har variabler, metoder og klasser beskrivende navn?		- Svært mye bruk av variabelnavn som ikke gjenspeiler data de representerer/holder. - Navn på metoder gjenspeiler i ingen eller liten grad hva metoden gjør.	- Gjennomgående gode variabelnavn som gjenspeiler data de representerer/holder. - Gjennomgående gode navn på metoder som gjenspeiler hva metoden gjør. - Klassenavn beskriver godt rollen/ansvaret til klassen	- Svært gode variabelnavn som gjenspeiler data de representerer/holder - Svært gode navn på metoder som gjenspeiler hva metoden gjør - Alle klassenavn beskriver godt rollen/ansvaret til klassen
Er klassene gruppert i en logisk pakkestruktur?	Benytter ikke pakker	Enkel pakkestruktur (f.eks. alle klasser i en pakke)	Grei pakkestruktur som separerer klassene i.h.t. klassisk lagdeling (presentasjon, forretningslogikk og data)	Svært god og fornuftig pakkestruktur, der klasser grupperes i pakker med gode beskrivende navn som tydelig gjenspeiler rolle/ansvarsområde

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Har koden god struktur, med løse koblinger og høy kohesjon?		• Det er tett kobling mellom klasser som ikke burde hatt kobling (high coupling) • Mange metoder som utfører flere oppgaver (low cohesion) • Mange klasser med flere roller/ansvar	• Generelt god løs kobling mellom klasser som ikke burde hatt kobling (low coupling) • De fleste metoder utfører jevnt over én enkelt oppgave (high cohesion). Der en metode må utføre flere del-oppgaver, benyttes delegering til sub-metoder. • De fleste klasser har en klart definert rolle/ansvar	• Gjennomgående svært god løs kobling mellom klasser som ikke burde hatt kobling (low coupling) • Samtlige metoder utfører en enkelt oppgave (high cohesion). Der en metode må utføre flere del-oppgaver, benyttes delegering til sub-metoder. • Samtlige klasser har en klart definert rolle/ansvar • Utstrakt grad av å bruke interface eller superklasser for variabler, i stedet for konkrete klasser, der arv er i bruk. (returnerer List<> og ikke ArrayList<> f.eks.. bidrar til løsere kobling).
Bonus				
Dette er en "pott" som vi kan tildele studenter som har utmerket seg spesielt, eller som kriteriene over ikke helt fanger. Disse poengene kommer på toppen av maks (kan da oppnå 110%).				

Del 3 – Rapport (30%)

Kandidatene har fått utlevert en rapportmal. I malen er det gitt en mengde *hjelpetekst* for å hjelpe kandidaten til å forstå hva som er forventet i de ulike kapitlene i rapporten. Malen baserer seg på strukturen som ofte benyttes i vitenskapelige artikler og i Bachelor-rapporter.

Hovedfokuset i rapporten skal ligge i kapitlene Resultat og Diskusjon.

KRITERIUM	Ikke bestått (F)	Svak (E, D)	Middels (C)	Utmerket (B, A)
Struktur	Kaos	Elementer mangler eller er plassert på ulogiske steder. Rapporten har figurer som er uleselige.	Grei struktur, fin rød tråd, ting er beskrevet der de hører hjemme. Følger godt sammenhengen mellom teori, metode, resultat og diskusjon.	Svært god struktur med referanser til kilder. God referanseliste som også benyttes (refereres til).
Innledning/Kravspesifikasjon	Mangler	Vanskelig å skjønne hva som skal utvikles basert på beskrivelsen. Bare kopiert oppgaveteksten i sin helhet, eller delvis.	Har beskrevet de viktigste funksjonelle kravene	Utfyllende med f.eks. - use case-diagrammer - og beskrivelser - andre typer krav (systemkrav, ikke-funksjonelle krav)
Teori (kort)	Mangler	Svært mangelfullt. Nevner teori som ikke benyttes i løsningen. Lang utledning av teori som ikke er relevant.	God kort og konsis beskrivelse av teori som er benyttet for å løse oppgaven. Noe mangelfullt, eller noe vel detaljert, men jevnt over bra.	Beskriver kort og konsist kun teori som er relevant for løsningen. Refererer til kilder for detaljer.
Design/Implementasjon (Resultat)	Mangler	Utydelig, mangler diagrammer og skisser. Ingen skjermdumper av endelig løsning.	Beskriver godt hvordan løsningen er designet. Benytter diagrammer (klassediagrammer m.m.) på en god måte. Wireframes for å beskrive design av GUI. Skjermdump av endelig GUI. Kode-eksempler der dette er nyttig for å påpeke godt designede løsninger.	Svært god beskrivelse av løsningen. Ekstra elementer som f.eks. komponentdiagram, sekvensdiagram, aktivitetsdiagram for utvalgte funksjoner.
Prosess (Resultat)	Mangler	Knappt beskrevet og uten illustrasjoner.	Godt beskrevet hvordan gruppen har jobbet i prosjektet.	Svært godt beskrevet (men kort og konsist) hvordan gruppen har jobbet sammen (hvordan ulike verktøy har blitt benyttet, hvordan tilbakemeldingssamtalene ble gjennomført osv.)

Refleksjon (Diskusjon)	Mangler	Mangler refleksjon om én eller flere av eget arbeid, læring og resultat.	Grei refleksjon over løsningen, arbeid, læring og resultat. Viser konkret til hvordan deler i løsningen oppfyller designkrav basert på teoriene.	Svært god refleksjon rundt endelig løsning i forhold til valgt metode og bakgrunnsteorier. Beskriver også forslag til fremtidig videreutvikling/endring/refaktoring.
------------------------	---------	--	--	---

Illustrasjoner

Bildet på forsiden er utarbeidet av pch.vector/Freepik.

Vedlegg

Mappebeskrivelsen er levert ut til kandidatene i tre deler som bygger på hverandre. Den fullstendige mappebeskrivelsen er vedlagt.